

세이브 로드 — 구현 기획 (단계별)

날짜: 2026-05-01 **선행:** 프로젝트 전반 메커니즘.md , 출시까지 — 남은 병목 정리.md

§0. 확정 사항

- 위치: `user://save/slot_0.json` (단일 슬롯). 다중 슬롯·자동저장은 다음 차로.
- 데이터 출처: RunManager 의 `big_run_data` + `run_data` 통째로 dump.
- 방식: GameManager 가 RunManager 의 dict 를 직접 read/write. 삼항 §3 의 cross-manager 직접 접근을 영속화 한정 의도적 허용 — 코드 한 줄 주석으로 명시.
- 저장 시점 게이팅: `phase == "map"` 만 허용. 전투·이벤트·연구 중 저장 X.
- JSON 직렬화: `JSON.stringify` / `JSON.parse_string` 로 충분 (run/big_run dict 가 primitive 만 담음).
- 버전: `version: 1` 박아둠. 호환 깨질 변경 시 증가.
- 로드 후: `RunManager.state_changed.emit()` + `GameManager.app_state_changed.emit()` 둘 다 발화.
- 물리 폴더: `user://save/` 이미 생성됨 (`~/Library/Application Support/Godot/app_userdata/` 새 게임 프로젝트/save/).

§1. 단계 표

단계	산출물	검증
S1	경로 상수 + <code>DirAccess.make_dir_absolute</code> 자동생성	헤드리스 (path 존재 확인)
S2	<code>GameManager.save()</code> 구현	헤드리스 (시물 → save → 파일 존재·JSON 파싱)
S3	<code>GameManager.load_save()</code> 구현	헤드리스 (save → reset → load → 상태 일치)
S4	UI 배선 — title 의 "불러오기" 실 구현 + 게임 중 "저장" 버튼	실 플레이 (스크린샷 1~2컷)
S5	통합 실 플레이 검증	실 플레이 + 회귀 96 전부 PASS 유지

(S6 다음 차로: 자동저장 / 다중 슬롯 / 백업본 / 마이그레이션)

MVP = S1 ~ S5. 예상 총 1일.

§2. Stage S1 — 기반

작업 범위

`game_manager.gd` 에 상수 + dir ensure:

```
const SAVE_DIR := "user://save/"
const SAVE_PATH := "user://save/slot_0.json"
const SAVE_VERSION := 1

func _ready() -> void:
    DirAccess.make_dir_absolute(SAVE_DIR) # 이미 있으면 무탈
```

검증 시나리오

`test/test_save_load.{gd,tscn}` 신설. 첫 섹션: - `main.tscn` 인스턴스 - `await`
`get_tree().process_frame` - `DirAccess.dir_exists_absolute("user://save/")` 검증

§3. Stage S2 — `save()` 구현

작업 범위

```
# 직접 dict 조작 - 삼항 §3 의 cross-manager write 를 영속화 한정 의도적 허용.
# (영속화는 GameManager 의 책임, RunManager 는 데이터만 소유. dict 스키마 진화 시
# 인터페이스 wrapper 갱신 부담 회피.)
func save() -> bool:
    if RunManager.run_data.is_empty():
        push_warning("save: 활성 런 없음")
        return false
    if RunManager.run_data.get("phase", "") != "map":
        push_warning("save: phase != 'map' - 안전 시점에서만 저장")
        return false
    var payload := {
        "version": SAVE_VERSION,
        "saved_at": Time.get_datetime_string_from_system(),
        "big_run_data": RunManager.big_run_data,
        "run_data": RunManager.run_data,
    }
    var f := FileAccess.open(SAVE_PATH, FileAccess.WRITE)
    if f == null:
        push_warning("save: 파일 열기 실패")
        return false
```

```
f.store_string(JSON.stringify(payload, " "))
return true
```

검증 시나리오

```
GameManager.start_run() + intro 소비 (phase = map)
RunManager.move_to_node(2) # 적 노드 - battle_preview 진입은 일단 무시
RunManager.run_data["phase"] = "map" # 강제 세팅 (테스트 우회)

# 정상 저장
ok = GameManager.save()
check ok == true
check FileAccess.file_exists(SAVE_PATH)

# 파일 내용 JSON 파싱 가능
content = read SAVE_PATH
data = JSON.parse_string(content)
check data.version == 1
check data.has("big_run_data") and data.has("run_data")
check data.big_run_data.meta.big_run_count == 0

# phase != map 시 거부
RunManager.run_data["phase"] = "event"
ok = GameManager.save()
check ok == false # 거부됨
```

체크 ~10 항목.

§4. Stage S3 — load_save() 구현

작업 범위

```
# 직접 dict 조작 - S2 와 동일한 의도적 §3 허용.
func load_save() -> bool:
    if not FileAccess.file_exists(SAVE_PATH):
        return false
    var f := FileAccess.open(SAVE_PATH, FileAccess.READ)
    if f == null:
        return false
    var content := f.get_as_text()
    var data = JSON.parse_string(content)
    if data == null or not (data is Dictionary):
        push_warning("load: JSON 파싱 실패")
        return false
    if not data.has("version"):
```

```

        push_warning("load: 알 수 없는 형식")
        return false
    RunManager.big_run_data = (data.get("big_run_data", {}) as Dictionary).duplicate()
    RunManager.run_data = (data.get("run_data", {}) as Dictionary).duplicate(true)
    RunManager.state_changed.emit()
    app_phase = "in_run"
    app_state_changed.emit()
    return true

func has_save() -> bool:
    return FileAccess.file_exists(SAVE_PATH)

```

검증 시나리오

```

# 시나리오 1: save → reset → load → 상태 일치
GameManager.start_run() + intro 소비
RunManager.move_to_node(5) # regression 발화
EventManager.advance_line() # 종료
# 이제 seen_events.regression_speech = 1, seen_events.intro_speech = 1
# big_run_count = 0 (아직 회귀 X), current_node_id = 5

snapshot = {
    seen_events: deep copy,
    body_hp: ...,
    current_node_id: 5,
}

GameManager.save()
GameManager.return_to_title()
check RunManager.run_data.is_empty()

GameManager.load_save()
check RunManager.run_data.get("current_node_id") == 5
check RunManager.big_run_data["seen_events"].get("regression_speech") == 1
check RunManager.run_data.get("body_hp") == snapshot.body_hp
check GameManager.app_phase == "in_run"

# 시나리오 2: 세이브 없음 / 잘못된 파일
delete file
ok = GameManager.load_save()
check ok == false

# 잘못된 JSON
write garbage to file
ok = GameManager.load_save()
check ok == false # 거부

# version 키 없는 파일

```

```
write {"big_run_data": {}, "run_data": {}}
ok = GameManager.load_save()
check ok == false
```

체크 ~15 항목.

§5. Stage S4 — UI 배선

작업 범위

title_screen의 "불러오기" 버튼: - `game_ui.gd::_on_load_pressed` 가 현재 `print("[title] 불러오기: 아직 구현되지 않음")` 스텝. - `GameManager.load_save()` 호출로 교체. - 버튼 `enabled` 상태: `GameManager.has_save()` 결과로 토글 (없으면 회색).

게임 중 "저장" 버튼: - `run_ui.gd`의 우상단 영역 (회귀·잔액 라벨 근처)에 신규 버튼. - `visibility: phase == "map"` 만. - 클릭 → `GameManager.save()`. 결과 toast 또는 작은 피드백 라벨 (1~2초 후 사라짐).

검증 시나리오

실 플레이 (스크린샷 캡처): 1. 새 게임 → 저장 → 종료 → 재시작 → 불러오기 → 같은 위치에서 시작 2. 저장 시점 표시 (`saved_at`)가 title 화면 어디에 있으면 좋음 (다음 차로) 3. 저장 안 된 상태에서 "불러오기" 회색 처리 확인

스크린샷 ~3컷: - `s4_01_save_button_visible`: 저장 버튼이 우상단에 보임 - `s4_02_after_save`: 저장 직후 (피드백) - `s4_03_loaded_state`: 로드 후 동일 상태

§6. Stage S5 — 통합 실 플레이 검증

시나리오 (수동, 5분 분량)

1. 새 게임 시작 → intro 발화 → "시작합니다." → "환경을 확인합니다." 진행
2. 노드 5 (event) → "회귀합니다." 진행
3. 노드 6 (repair) → 팔 수리 선택 → "팔 수리 완료." 진행
4. 노드 4 (research) 진입 → 연구 옵션 1개 적용
5. 회귀 종료 → 새 회차 시작 (회귀 카운트 = 1)
6. 저장
7. 타이틀로 → 게임 종료 → 재실행 → 불러오기
8. 회귀 카운트 1 + 적용된 연구 효과 + 새 회차 진행 상태 모두 그대로 복원 확인

자가점검 체크리스트

- [] 저장 후 종료 → 재시작 → 불러오기 → 상태 100% 복원

- [] 회귀 카운트, seen_events, research_data, body_hp/max, arm 인스턴스 (cards 변경분 포함), equipped_arms 모두 복원
- [] phase = "map" 일 때만 저장 가능
- [] 저장 없이 "불러오기" 클릭 시 무탈 (false 반환, 상태 변경 없음)
- [] 잘못된 JSON 파일 직접 만들고 불러오기 시도 → 거부, 게임 무탈

§7. 코드 변경 파일 (예상)

파일	변경
game_manager.gd	<code>_ready</code> 의 dir ensure, <code>save()</code> / <code>load_save()</code> / <code>has_save()</code> 신설. ~50줄 추가.
game_ui.gd	<code>_on_load_pressed</code> 가 실제 호출. 버튼 enabled 토글. ~10줄.
run_ui.gd	"저장" 버튼 build-refresh + 피드백 라벨. ~30줄.
test/test_save_load. {gd, tscn}	신설 검증. ~150줄.

기존 RunManager / EventManager 변경 0. 회귀 96/96 그대로 통과 예상.

§8. 진입 순서·예상 분량

단계	작업	누적 시간
S1	경로 상수 + dir ensure	~15분
S2	save() 구현 + 헤드리스 검증	~2시간
S3	load_save() 구현 + 헤드리스 검증	~3시간
S4	UI 배선 + 스크린샷	~2시간
S5	실 플레이 통합	~30분
	합계	~1일

§9. 다음 차로 (S6+)

MVP 종료 후 자연스러운 확장:

1. 자동저장 — 회귀 시 / 큰 phase 변경 시 / 종료 시 `auto.json` 별도 슬롯에.
 2. 다중 슬롯 — `slot_0`, `slot_1`, `slot_2` + 슬롯 선택 UI.
 3. 백업본 — 저장 시 직전본을 `backup_0.json` 으로 보존 (사고 방지).
 4. 저장 메타 정보 표시 — title 의 "불러오기" 버튼에 `saved_at`, 회귀 카운트 등 미리보기.
 5. 버전 마이그레이션 — `SAVE_VERSION` 증가 시 변환 로직. 출시 후 패치 단계.
 6. 수동 저장 → 안전 시점 자동만 으로 정책 변경 검토.
-

§10. 즉시 결정 (한 가지)

현재 게임 중 "저장" 버튼 위치: 우상단 회귀 라벨 근처 / 좌상단 / 별도 메뉴 (ESC 키 메뉴) 중 어느 쪽?

가장 단순 = 우상단 회귀 라벨 옆. 전체 메뉴는 다음 차로.

진입 신호 + §10 위치 결정 받으면 S1 부터 시작.